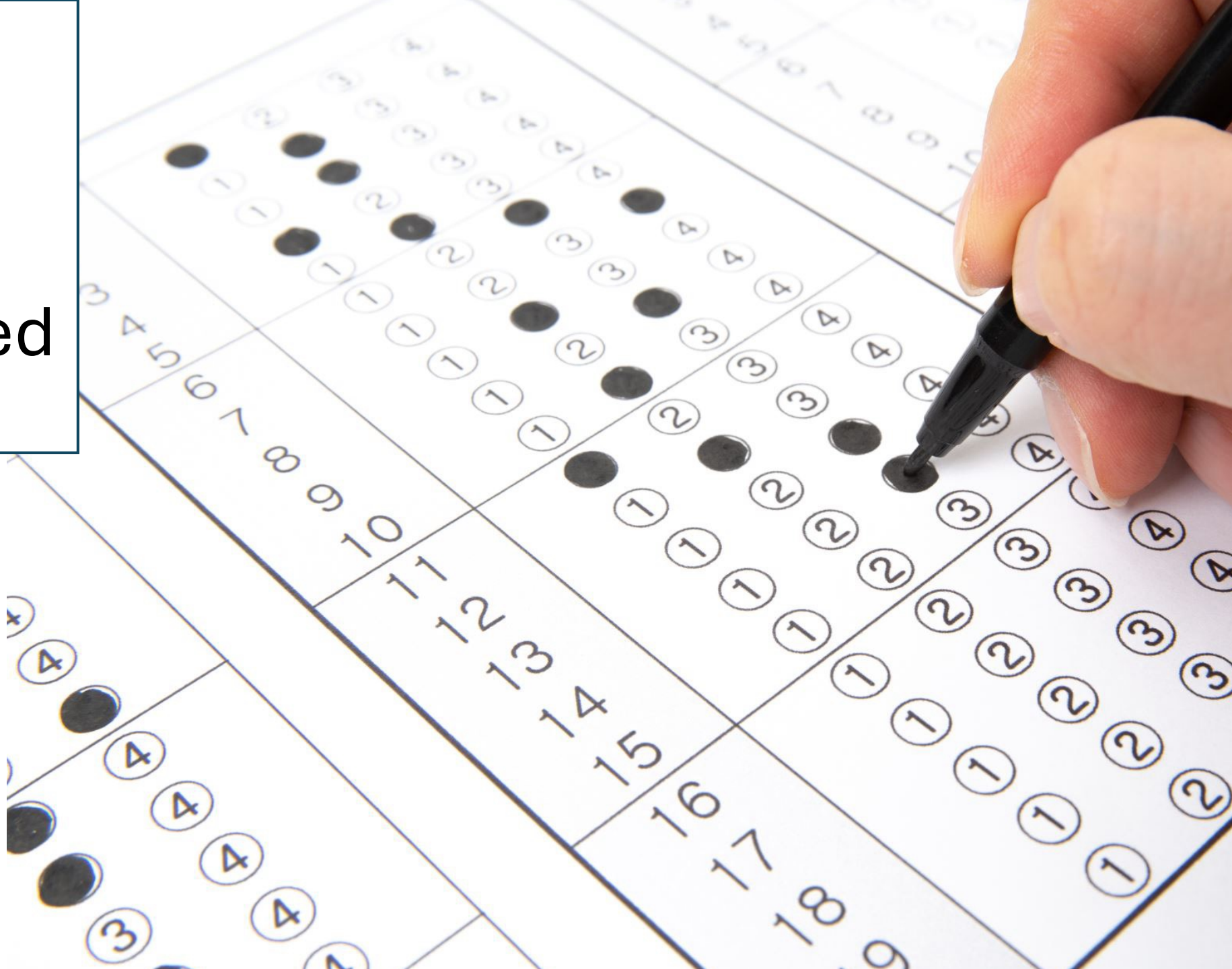
Predicting Proficiency: A Custom Weighted LightGBM-Based Approach

Jeremy Gomez
Kaggle: Jerm.gome



Pipeline Architecture

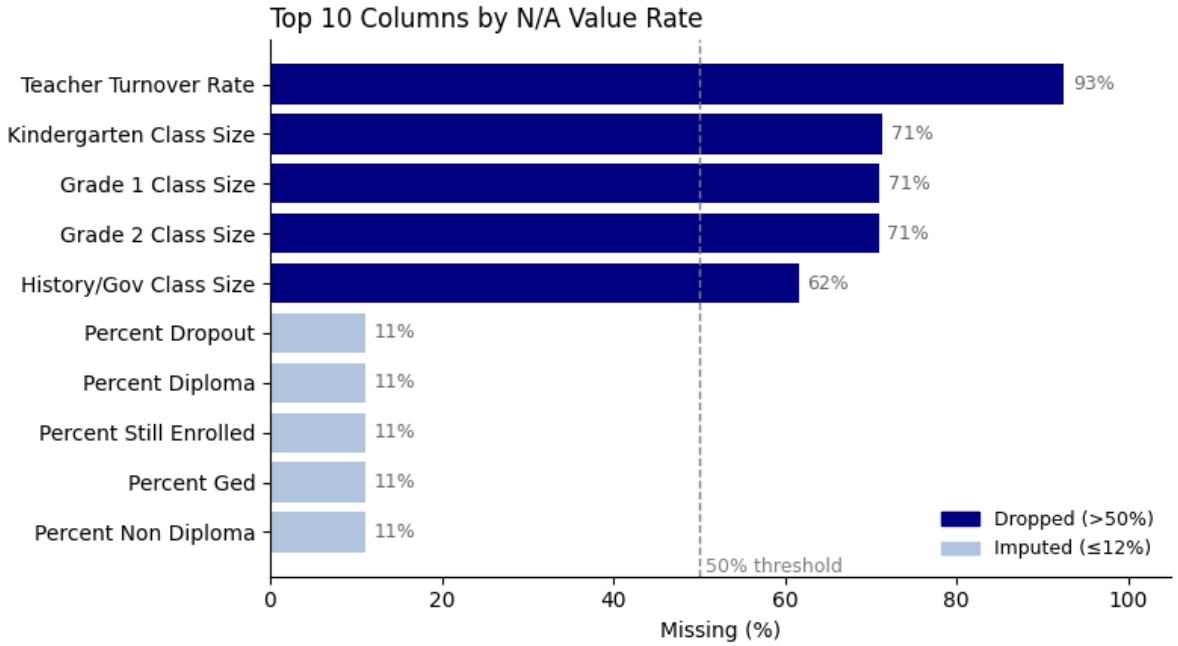


Data Reuse

- 80-20 Train/Test Split on cleaned, imputed, and feature engineered data
- 6-fold cross validation inside Optuna for hyperparameter tuning
- Final model retrained on all (train and test) labeled data

Data Analysis & Imputation

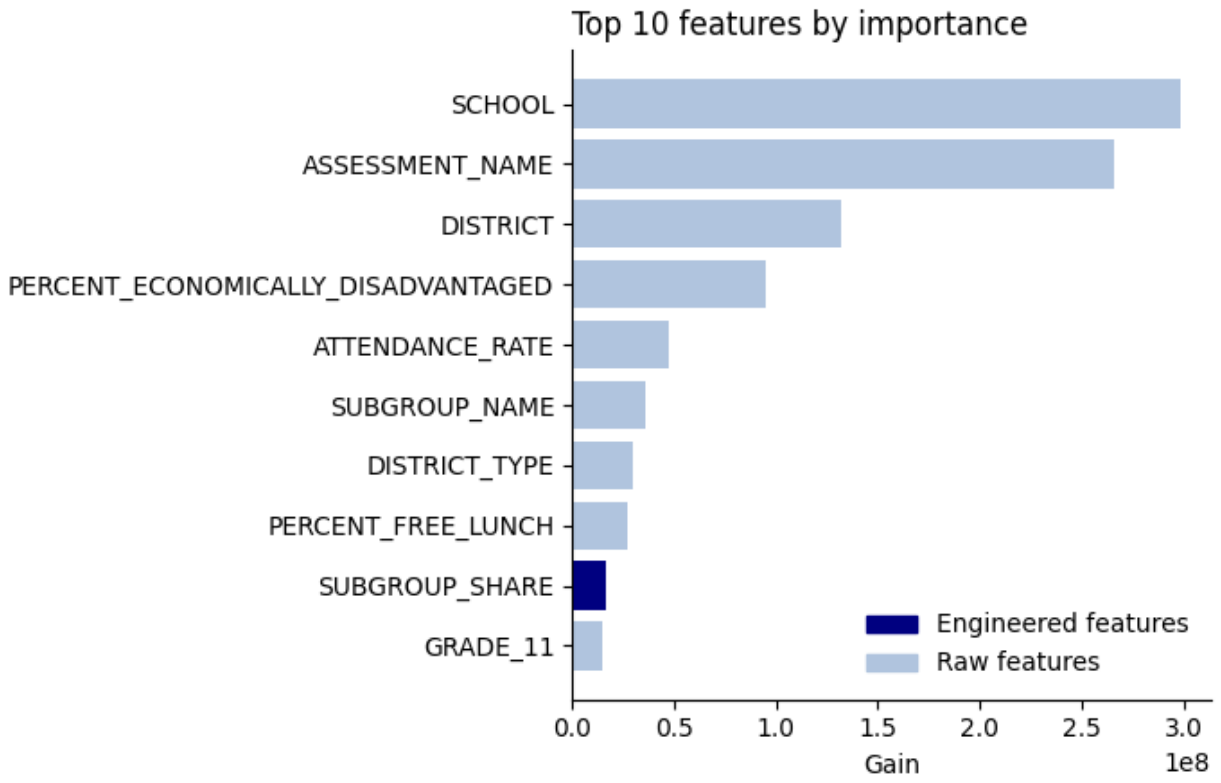
- 5 features had large (>50%) N/A value rates, these were dropped from the training set due to data reliability issues
- The remaining features had small (<12%) null value rates, these features were imputed using the median values of different observations from the same school, with statewide median values being used as a fallback



Feature Engineering

I added 5 features that contributed less than expected to my model

- Federal Funding Ratio (Federal Funding/Total Funding)
- Student to Teacher Ratio (# Students/# Teachers)
- Subgroup Share (Fraction of the school's student population taking the exam)
- Total Funding Per Pupil (Federal + Local Funding per pupil)
- Mean Class Size (Average of ELA, Math & Science class sizes)



LightGBM + ReLU-penalized loss

LightGBM was my chosen model because it is a decision tree-based model, tree-based models are suited to training data where there are:

- High cardinality categorical features (Schools, Districts, etc.)
- Nonlinear effects of features on the target category for each observation
- Large interaction effects between features

LGBM uses a default L2 loss which does not penalize predictions of less than 0% and more than 100%, instead I switched this loss function out for a scaled ReLU hinge penalty loss function which penalizes predictions that are lower than 0% and greater than 100%

Hyperparameter Tuning with Optuna

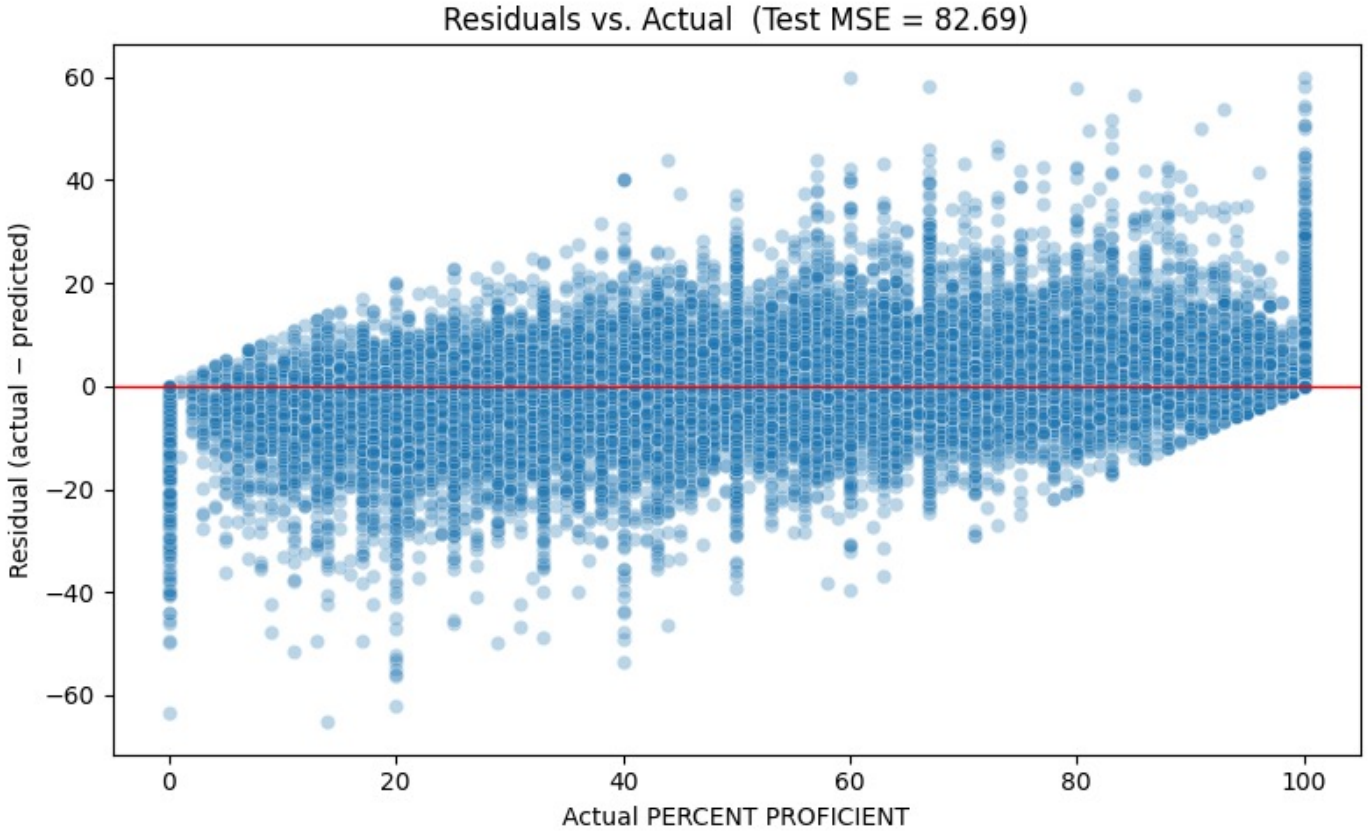
Optuna is an open-source library for automatic hyperparameter tuning of machine learning models such as LightGBM

- I tuned 11 Light GBM hyperparameters, a grid search method of tuning 11 hyperparameters would have n^{11} trials (where n is the number of possible values per hyperparameter)
- I used a 6-fold cross validation inside Optuna to set the hyperparameters for my Light Gradient Boost Model
- Optuna sets hyperparameters using a TPE (Tree-structured Parzen Estimator) sampler which proposes hyperparameter values after each trial using previously evaluated trials

Reflections and Room for Improvement

The next time I find myself competing in a prediction contest I intend to:

- Investigate stacking techniques, my residuals vs actual plot shows that I have significant bias in my estimated values. Especially near 0% and 100%
- Focus on the features. Spend more time thinking about how I could apply both model-specific and model-agnostic feature engineering to include more high contribution features to the model



Thank You